

JPA: Java Persistence API

version 2.1

Licence Pro 2014

Olivier Perrin

Université de Lorraine

Plan

ORM

Entités

Associations

Comportement

Requêtes

Mise en œuvre

ORM = Object Relational Mapping

Object/Relational Mapping

Beaucoup d'applications ont besoin d'une couche de persistance

- accès à des données persistantes stockées dans un SGBD
- gestion des données persistantes dans le SGBD

ORM résout la différence entre modèle Objet et modèle Relationnel

- les données sont dans la BDD relationnelle: orientée tables, nuplets, colonnes
- l'application travaille avec des classes, des objets, pas des nuplets ou des colonnes

Autres possibilités: Hibernate, JDO,...

JPA ?

Standard offert par JavaEE/JavaSE

Framework ORM

Persistance transparente

POJO

Respecte les concepts OO

JPA: les besoins

Simplifier le modèle de persistance

Modèle de persistance léger

Support de modélisation complexe

- héritage, polymorphisme

ORM standard et efficace

- optimisé pour les SGBD relationnels
- annotations standardisées

Requêtes

Plusieurs supports de persistance

- notion de *persistence unit* (persistence.xml)

JPA 2.1

Entités

Types de base

- Strings, integers, floats, decimals

Classes *embeddable* améliorées

- par exemple une adresse
- *embeddables* imbriquées; *embeddables* avec associations

Associations

- one-to-one, one-to-many/many-to-one, many-to-many
- collections en utilisant `java.util.Collection`, `Set`, `List`, ou `Map`
- paramétrage avec `@JoinColumn`, `@JoinTable`, ...

Héritage

- table simple, sous-classe, table par classe (optionnel)

JPA 2.1: nouveautés

Collections

- collections de strings, integers, floats, decimals, ...
- collections de classes *embeddable*

Listes triées de manière persistante

Support amélioré de Map

Options supplémentaires pour les associations

- clé étrangère one-to-many unidirectionnelle
- table de jointure pour one-to-one, one-to-many/many-to-one

JPQL

Gestion des caches

JPA 2.1: nouveautés (cont.)

Jointures avec conditions ON

Invocation de fonctions de la BDD

Mises à jour/suppression en série

Casts dans les clauses FROM et WHERE

Options pour la génération de schémas

JPA 2.1: nouveautés (cont.)

```
public List obtainActiveEmployeeCount() {
    TypedQuery<Object[]> qry = em.createQuery("SELECT j.title, count(e) "
        + "FROM Jobs j LEFT JOIN j.employees e ON e.status = 'ACTIVE' "
        + "WHERE j.salary >= 50000 GROUP BY j.title", Object[].class);

    List data = new ArrayList();
    if (!qry.getResultList().isEmpty()) {
        List<Object[]> tdata = qry.getResultList();
        for (Object[] t : tdata) {
            HashMap resultMap = new HashMap();
            resultMap.put("title", t[0]);
            resultMap.put("count", t[1]);
            data.add(resultMap);
        }
    }
    return data;
}
```

JPA 2.1: nouveautés (cont.)

```
SELECT e FROM Employee e WHERE FUNCTION('aboveTaxBracket', e.salary)
```

```
CriteriaBuilder builder = em.getCriteriaBuilder();  
CriteriaUpdate<Employee> q = builder.createCriteriaUpdate(Employee.class);  
Root<Employee> e = q.from(Employee.class);  
q.set(e.get("status"), "ACTIVE")  
    .where(builder.equal(e.get("status"), "INACTIVE"));  
Query criteriaUpd = em.createQuery(q);  
criteriaUpd.executeUpdate();
```

```
SELECT b.attr1, b.attr2  
FROM EntityA a JOIN TREAT(a.referenceToEntityB as EntityBType) b
```

Entités

JPA: annotations

Ensemble d'annotations permettant de définir les entités

- ▶ relations
- ▶ attributs
- ▶ clé
- ▶ associations
- ▶ ...

Deux possibilités pour définir le mapping

- ▶ annotations
- ▶ ou fichier de mapping

Entité

Plain Old Java Object (POJO)

- créée avec le mot-clé `new` comme toute autre instance Java

Peut contenir des attributs persistants ou non

- état non persistant accessible grâce à l'annotation `@Transient`

Peut étendre d'autres entités ou des classes qui ne sont pas des entités

Sérializable

- pas besoin de s'occuper des transferts d'objets

Mapping simple

Client

```
id: Long
prénom: String
nom: String
téléphone: String
email: String
age: Integer
dateNaissance: Date
```

Relationnel

```
@Entity
public class Client {
    @Id
    private Long id;
    private String prénom;
    private String nom;
    private String téléphone;
    private String email;
    private Integer age;
    private Date dateNaissance;
    ...
    // constructeurs/setter/getter
}
```

Java

Règles

Pour qu'une classe puisse être persistante, il faut

- qu'elle soit identifiée comme une entité (*entity*) en utilisant l'annotation `@java.persistence.Entity`
- qu'elle possède un attribut identifiant en utilisant l'annotation `@javax.persistence.Id`
- qu'elle ait un constructeur sans argument

```
@Entity
public class Client {
    @Id
    private Long id;
    private String prénom;
    private String nom;
    private String téléphone;
    private String email;
    private Integer age;
    private Date dateNaissance;
    // constructeurs/setter/getter
}
```

Clé primaire

Une entité doit avoir un attribut qui correspond à la clé primaire de la table associée

La valeur de cet attribut ne doit jamais être modifiée

L'attribut clé primaire est désigné par l'annotation `@Id`

Pour une clé composite, on utilise `@EmbeddedId` ou `@IdClass`

Le type de la clé primaire doit être d'un des types suivants:

- type primitif Java
- classe qui enveloppe un type primitif
- `java.lang.String`
- `java.util.Date`
- `java.sql.Date`

Clé primaire (cont.)

Possibilité de générer automatiquement les clés

Si la clé est de type numérique

- ▶ `@GeneratedValue` indique que la clé sera automatiquement générée par le SGBD
- ▶ l'annotation peut avoir un attribut **strategy** qui indique comment la clé sera générée
 - **AUTO**: le SGBD choisit (valeur par défaut)
 - **SEQUENCE**: il utilise une séquence SQL
 - **IDENTITY**: il utilise un générateur de type IDENTITY (auto increment dans MySQL par exemple)
 - **TABLE**: il utilise une table qui contient la prochaine valeur de l'identificateur

Attributs

Paramétrage en utilisant l'annotation `@Column`

Attributs de `@Column`

- ▶ **name()**: nom de l'attribut
- ▶ **unique()**: la valeur est-elle unique ?
- ▶ **nullable()**: accepte une valeur nulle ?
- ▶ **insertable()**: autorise ou non l'attribut à être mis à jour
- ▶ **columnDefinition()**: définition DDL de l'attribut
- ▶ **table()**: lorsque l'attribut est utilisé dans plusieurs tables
- ▶ **length()**: longueur max
- ▶ **precision()**: précision pour les valeurs numériques

Relation Clients revisitée

```
@Entity
@Table(name = "rClient")
public class Client {
    @Id @GeneratedValue(strategy = GenerationType.AUTO) private Long id;
    @Column(nullable = false) private String prénom;
    @Column(nullable = false, length = 30) private String nom;
    @Column(length = 15) private String téléphone;
    @Column(name = "MAIL") private String email;
    private Integer age;
    ...
}
```



desc rClient		
Field	Type	Null
ID	bigint(20)	NO
PRENOM	varchar(255)	NO
NOM	varchar(30)	NO
TELEPHONE	varchar(15)	YES
MAIL	varchar(255)	YES
AGE	int(11)	YES

Annotations supplémentaires

@Temporal

- ▶ on peut ajouter l'annotation pour préciser le type d'un attribut
- ▶ `@Temporal(TemporalType.DATE) private Date dateNaissance;`
- ▶ on a le choix entre DATE, TIME, TIMESTAMP

@Transient

- ▶ toutes les informations d'une classe n'ont pas besoin d'être persistantes
- ▶ par exemple, l'âge
- ▶ `@Transient private Integer age;`

Clé composite

Relation existante avec clé multi-attributs

Relation avec association N-M (*Many-to-Many*)

2 possibilités

- ▶ `@IdClass`: correspond à plusieurs attributs *Id* dans la classe entité
- ▶ `@EmbeddedId` et `@Embeddable`: un seul attribut *Id* dans la classe entité

Dans les 2 cas, la clé doit être représentée par une classe Java

- ▶ les attributs correspondent aux composants de la clé
- ▶ la classe doit être publique
- ▶ la classe doit avoir un constructeur sans paramètre
- ▶ la classe doit être sérializable et redéfinir `equals` et `hashCode`

@EmbeddedId

Correspond au cas où la classe entité comprend un seul attribut annoté @EmbeddedId

La classe clé primaire est annotée par @Embeddable

```
@Entity
public class Client {
    @EmbeddedId
    private ClientPK clientPK;
    ...
}
```

```
@Embeddable
public class ClientPK {
    private String nom;
    private Date dateNaissance;
    ...
}
```

@IdClass

Correspond au cas où la classe entité comprend plusieurs attributs annotés par @Id

La classe entité est annotée par @IdClass qui prend en paramètre le nom de la classe clé primaire

La classe clé primaire n'est pas annotée

- ses attributs ont les mêmes noms et mêmes types que les attributs annotés @Id dans la classe entité

```
@Entity
@IdClass(ClientPK)
public class Client {
    @Id private String nom;
    @Id private Date dateNaissance;
    ...
}
```

```
public class ClientPK {
    private String nom;
    private Date dateNaissance;
    ...
}
```

Associations

Associations

Une association peut être uni- ou bi-directionnelle

Cardinalités: 1-1, 1-N, N-1, M-N

Elles sont définies grâce à une annotation sur la propriété correspondante

```
// Association entre Client et Commandes
@OneToMany
public Collection<Commande> getCommandes() {
    return commandes;
}
```

Support de Collection, Set, List, Map

Associations bidirectionnelles

Le développeur est responsable de la gestion des deux bouts de l'association

Un des bouts est le propriétaire de l'association

Propriétaire

- pour les associations autres que M-N, ce bout correspond à la table qui contient la clé étrangère qui traduit l'association
- pour les associations M-N, le développeur choisit le bout propriétaire (de manière arbitraire)
- l'autre bout (non propriétaire) est qualifié par l'attribut **mappedBy** qui donne le nom de l'association correspondante dans le bout propriétaire

Association 1-1

Annotation @OneToOne

Représente la clé étrangère dans la table

```
// Association entre Client et Adresse  
@OneToOne  
public Adresse getAdresse() {  
    ...  
}
```

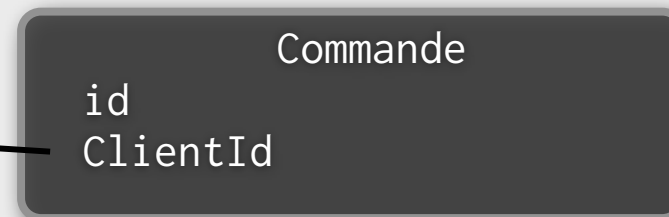
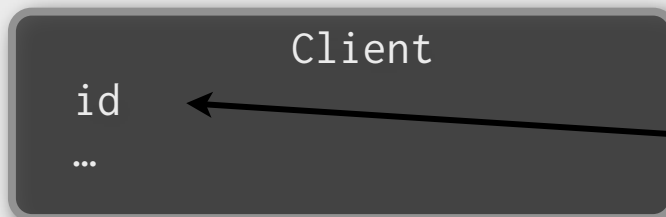
Associations 1-N et N-1

Annotations @OneToMany et @ManyToOne

Représentée par une clé étrangère dans la table qui correspond au côté propriétaire (obligatoirement le côté Many)

```
@Entity
public class Client {
    @Id
    private int id;
    ...
    @OneToMany(mappedBy = "c")
    Set<Commande> commandes;
}
```

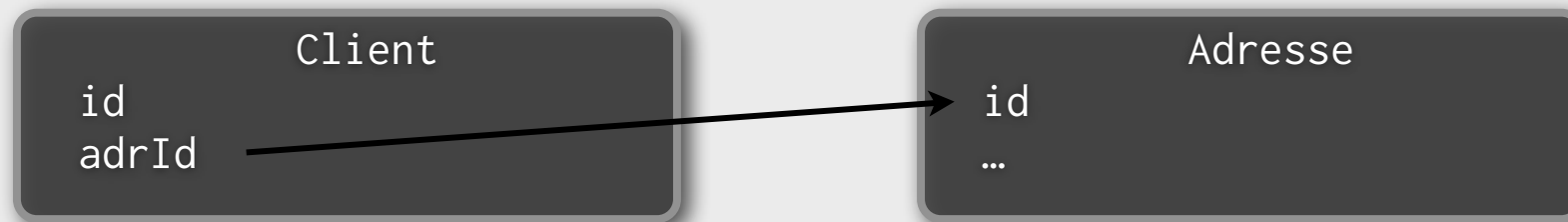
```
@Entity
public class Commande {
    @Id
    private int id;
    ...
    @ManyToOne
    Client c;
}
```



Associations 1-N et N-1 (cont.)

@ManyToOne

```
@Entity
public class Client {
    @Id
    private int id;
    ...
    @ManyToOne
    Adresse adr;
}
```



Associations 1-N et N-1 (cont.)

Exemple 1

- Entité Client
 - `@OneToMany(mappedBy="client")`
 - `public Set<Commande> commandes;`
- Entité Commande
 - `@ManyToOne public Client client;`

Exemple 2

- Entité Client
 - `@OneToMany(mappedBy="client", cascade=ALL)`
 - `public Set<Commande> commandes;`
- Entité Commande
 - `@ManyToOne`
 - `@JoinColumn(name="client_id")`
 - `public Client client;`

Associations M-N

Annotation @ManyToMany

Représentée par une table association

Les valeurs par défaut

- ▶ le nom de la table association est la concaténation des 2 tables, séparées par _
- ▶ les noms des attributs clés étrangères sont les concaténations de la table référencée, de _ et de l'attribut Id de la table référencée

Associations M-N (cont.)

```
@Entity
public class Client {
    @Id
    private int id;
    ...
    @ManyToMany
    Collection<Telephone> noTels;
}
```

```
@Entity
public class Telephone {
    @Id
    private int id;
    ...
    @ManyToMany(mappedBy="noTels")
    Collection<Client> clients;
}
```



Associations M-N (cont.)

Si le mapping par défaut ne convient pas, on peut le surcharger avec l'annotation **@JoinTable**

L'autre côté doit toujours comporter l'attribut **mappedBy**

```
@Entity
public class Client {
    @Id
    private int id;
    ...
    @ManyToMany
    @JoinTable(name="Client_Telephone",
        joinColumns=@JoinColumn(name="clients_id"),
        inverseJoinColumns=@JoinColumn(name="noTels_id"))
    Collection<Telephone> noTels;
}
```

Associations M-N (cont.)

@JoinTable

- ▶ donne des informations sur la table association qui va représenter l'association
- ▶ attribut **name** donne le nom de la table
- ▶ attribut **joinColumns** donne les noms des attributs de la table qui référencent les clés primaires du côté propriétaire de l'association
- ▶ attribut **inverseJoinColumns** donne les noms des attributs de la table qui référencent les clés primaires du côté qui n'est pas propriétaire de l'association

Identificateurs dérivés

En JPA 1.0

- identificateurs qui sont liés à une association
- nécessitent un attribut additionnel de clé étrangère
- mapping dupliqué

```
@Entity
public class Part {
    @Id Integer partNo;
    @Column (name="SUPP_ID")
    @Id Integer suppId;
    ...
    @ManyToOne
    @JoinColumn(name="SUPP_ID",insertable=false, updatable=false)
    Supplier supplier;
    ...
}
```

Identificateurs dérivés (2)

En JPA 2.1

- les identificateurs sont dérivés des associations

```
@Entity
public class Supplier {
    @Id long suppId;
    String name;
    ...
}

public class PartId {
    String partNo; // mapping vers nom de l'attribut @Id
    long supp;     // mapping vers nom de l'attribut @Id et type de Supplier PK
}

@Entity
@IdClass(PartId.class)
public class Part {
    @Id String partNo;
    @Id @ManyToOne Supplier supp;
    ...
}
```

Collections de types simples

```
@Entity
public class Personne {
    @Id protected String nss;
    protected String nom;
    protected Date dateNaissance;
    ...
    @ElementCollection
    protected Set<String> surnoms;
}
```

```
@Entity
public class Personne {
    @Id protected String nss;
    protected String nom;
    protected Date dateNaissance;
    ...
    @ElementCollection
    @CollectionTable(name="ALIAS")
    protected Set<String> surnoms;
}
```

Collections de types *embeddable*

```
@Embeddable
public class Adresse {
    String rue;
    String ville;
    String cp;
    ...
}

@Entity public class Millionaire extends Personne {
    ...
    @ElementCollection protected Set<Adresse> maisonVacances;
    ...
}
```

Niveaux multiples

```
@Embeddable
public class Adresse {
    String rue;
    String ville;
    String cp;
    ...
}
```

```
@Embeddable public class InfoContact {
    @Embedded Adresse adresse;
    ...
}

@Entity public class Employe {
    @Id int empId;
    String nom;
    InfoContact contact;
    ...
}
```

Embeddable avec associations

```
@Embeddable
public class InfoContact {
    @Embedded Adresse adresse;
    @OneToMany Set<Telephone> telephones;
    ...
}

@Entity public class Employe {
    @Id int empId;
    String nom;
    InfoContact contact;
    ...
}
```

Listes triées

```
@Entity public class CarteCredit {  
    @Id long numCarte;  
    @OneToOne Personne proprioCarte;  
    ...  
    @OneToMany  
    @OrderColumn  
    List<TransactionCarte> transactions;  
}
```

Maps

```
@Entity public class VideoStore {  
    @Id Integer storeId;  
    Adresse emplacement;  
    ...  
    @ElementCollection Map<Film, Integer> stock;  
}  
  
@Entity public class Film {  
    @Id String titre;  
    @String metteurEnScene;  
    ...  
}
```

Comportement

Comportement

Persistence des entités associées

- ▶ lorsqu'un objet *o* est rendu persistant, les objets référencés par *o* devraient être rendus persistants
- ▶ persistance par transitivité
- ▶ comportement logique, mais...
- ▶ complexe !
 - cohérence
 - par exemple, que se passe-t-il si un objet supprimé est référencé par un autre objet ?
- ▶ JPA n'effectue pas de persistance par transitivité par défaut
 - plus de souplesse
 - possibilité d'avoir la persistance par transitivité avec **cascade**

Comportement (cont.)

Attribut cascade

- ▶ permet de propager les effets d'une opération sur les entités associées
- ▶ valeurs possibles:
 - PERSIST, REMOVE, REFRESH, MERGE, ALL
- ▶ `@OneToMany(cascade=CascadeType.PERSIST)`

Par défaut, aucune opération n'est appliquée transitivement

En JPA 2.0, suppression automatique des orphelins

- ▶ `@OneToMany(cascade=CascadeType.PERSIST, orphanRemoval=true)`

Comportement (cont.)

Récupération des entités associées

- lorsqu'une entité est récupérée (requête), est-ce qu'on récupère aussi les entités associées ?
- et si on récupère les entités associées, est-ce que l'on récupère les entités associées des entités associées ?
- et si... !

JPA laisse le choix de récupérer ou non les entités associées

- on peut choisir le mode de récupération: LAZY ou EAGER
- **EAGER**: immédiat
- **LAZY**: à la demande

Comportement (cont.)

LAZY

- pour les associations avec des hiérarchies importantes
- dans ce cas, l'entité associée n'est pas récupérée immédiatement
- JPA remplace l'entité par un proxy qui permettra de récupérer l'entité plus tard si besoin
- le proxy contient la clé primaire

Par défaut

- JPA est en mode EAGER pour I-I et N-I
- JPA est en mode LAZY pour I-N et M-N

Interfaces principales

EntityManagerFactory

- utilisée pour créer des entity managers
- une seule entity manager factory par persistence unit

EntityManager

- utilisé pour gérer le contexte de persistance
 - écriture/lecture des entités dans/de la base de données
- opérations: persist, remove, find, refresh, createQuery,...

Query, TypedQuery

- utilisées pour la configuration des requêtes, la définition des paramètres et l'exécution des requêtes

Requêtes JPQL

Requêtes

Requêtes statiques

- `@NamedQuery`, `@NamedNativeQuery`

Requêtes dynamiques

- définition de la requête à l'exécution

`EntityManager` permet de créer et exécuter les requêtes

- `createNamedQuery`, `createQuery`, `createNativeQuery`

Recherche par identifiant

`find` et `getReference` permettent de retrouver une entité en donnant son Id

Elles ont deux paramètres

- `Class<T>` pour indiquer le type de l'entité recherchée
- `Object` pour indiquer la clé primaire

```
public void supprimeCommande(Long commandeId) {  
    Commande cmde = entityManager.find(Commande.class, commandeId);  
    entityManager.remove(cmde);  
}
```

Requêtes statiques

Définition

```
@NamedQuery(name="clientParCodePostal",  
    query = "SELECT c FROM Client c WHERE  
    c.adresse.codePostal = :cp")  
@Entity public class Client {  
    ...  
}
```

Utilisation

```
// Utilisation  
public List rechercheClientParCodePostal(int cp) {  
    return em.createNamedQuery("clientCodePostal")  
        .setParameter("cp", cp)  
        .setMaxResults(20)  
        .get  
}
```

Requêtes statiques (2)

Paramétrage

```
@NamedQuery(  
    name="clientParNom",  
    queryString="SELECT c FROM Client c " +  
        "WHERE c.nom LIKE :nomClient"  
)
```

```
@PersistenceContext public EntityManager em;  
List clients =  
    em.createNamedQuery("clientParNom")  
        .setParameter("nomClient", "Marchand")  
        .getResultList();
```

Requêtes dynamiques

```
public List rechercheAvecNom (String nom) {  
    return em.createQuery (  
        "SELECT c FROM Client c " +  
        "WHERE c.nom LIKE :nomClient")  
        .setParameter("nomClient", nom)  
        .setMaxResults(10)  
        .getResultList();  
}
```

Polymorphisme

Toutes les requêtes sont polymorphes

Oui, et alors ?

Un nom de classe dans la partie FROM désigne aussi bien l'entité désignée que ses sous-classes !

Exemple

- ▶ `select count(a) from Articles as a`
- ▶ dénombre le nombre d'instances de la classe Articles et de toutes les sous-classes de Articles

Expressions de chemins

On peut utiliser ces expressions pour naviguer entre les entités

Si `e` est un alias pour `Employe`

- ▶ `e.departement` désigne le département d'un employé
- ▶ `e.projets` désigne la collection de projets auxquels participe l'employé

On peut chaîner les navigations à condition que la navigation précédente ne renvoie pas un ensemble

- ▶ `e.departement.nom` désigne le nom du département de l'employé
- ▶ `e.projets.nom` renvoie une erreur !

Jointures

Trois types

- jointure interne
- jointure externe
- jointure avec récupération des données en mémoire (join fetch)

Exemples

- `select p.nom from Employe e join e.projets p`
- `select d.nom from Employe e, Departement d where d=e.departement`
- `select e.departement.nom from Employe e`

Pas plus de 2 niveaux dans une expression de chemin pour une jointure

- `select e,p from Employe e join e.participations.projet p`
renvoie une exception
- `select e,p from Employe e join e.participations pa join pa.projet p` est ok

Requêtes avec clé composée

```
public class MagasinPK implements Serializable {  
    private String codeVille;  
    private String codeMagasin;  
  
    public void setCodeVille(String v) {  
        codeVille = v;  
    }  
    public String getMagasinVille() {  
        return (codeVille);  
    }  
    public void setCodeMagasin(String c) {  
        codeMagasin = c;  
    }  
    public String getCodeMagasin() {  
        return (codeMagasin);  
    }  
}
```

Requêtes avec clé composée (cont.)

```
@Entity
@IdClass(MagasinPK.class)
public class Magasin {
    @Id @Column(name="NomMagasin") public String magasinCode;
    @Id public String villeCode;
    ...
}
```

```
PersistenceContext EntityManager em;
MagasinPK pk = new MagasinPK();
pk.setCodeMagasin("CSIN05");
pk.setCodeVille("JRE07");
Magasin m = em.find(Magasin.class, pk);
```

Update, delete

```
UPDATE Employe e  
SET e.salaire = e.salaire * 1.1  
WHERE e.departement.nom = 'Informatique'
```

```
DELETE  
FROM Client c  
WHERE c.status = 'inactif'  
AND c.commandes IS EMPTY  
AND c.solde = 0
```

Les mises à jour doivent être exécutées dans une transaction

Mise en œuvre

À partir d'un client JavaSE

```
public static void main(String[] args) {  
    EntityManagerFactory emf =  
        Persistence.createEntityManagerFactory("EmployeService");  
    EntityManager em = emf.createEntityManager();  
    Collection employes =  
        em.createQuery("SELECT e FROM Employe e").getResultList();  
    ...  
}
```

Entity Manager

C'est lui qui est chargé de gérer le cycle de vie des entités
Équivalent de *Hibernate Session* ou *JDO PersistenceManager*

4 opérations principales

- ▶ **persist()**
 - insère une entité dans la base
- ▶ **remove()**
 - supprime une entité de la base
- ▶ **merge()**
 - synchronise l'état des entités détachées
- ▶ **refresh()**
 - recharge l'état des entités à partir de la base

persist(), merge()

persist()

```
public Commande creerCommande(Client c) {  
    Commande cmde = new Commande(c);  
    // Après l'appel à persist(), l'état de l'entité passe  
    // transient à managed.  
    // Au prochain commit, l'instance sera insérée dans la  
    // base de données  
    entityManager.persist(cmde);  
    return cmde;  
}
```

merge()

- la méthode **merge()** retourne une copie gérée (*managed*) de l'entité (préalablement détachée)
- les modifications sur l'état persistant sont appliquées à l'instance gérée

Agir sur le cycle de vie d'une entité

Une application peut être notifiée avant ou après chaque action sur une entité (chargement, persistance, mise à jour, suppression) grâce aux méthodes callback

Annotations

- ▶ **PrePersist/PostPersist**
 - avant ou après que l'objet soit persistant
- ▶ **PreUpdate/PostUpdate**
 - avant ou après la mise à jour des attributs de l'objet
- ▶ **PreRemove/PostRemove**
 - avant ou après la suppression de l'objet
- ▶ **PostLoad**
 - après que l'objet ait été chargé depuis la base de données

Entités détachées

Entité qui a sa propre existence mais dont la synchronisation avec la BD n'est pas garantie

Exemple

- ▶ une entité est détachée
- ▶ elle est transmise à la couche cliente et modifiée par celle-ci
- ▶ l'entité est rattachée afin que son nouvel état soit enregistré dans la BD

L'entité doit implanter ***Serializable***

Elle devient détachée lors d'un **commit/rollback/close**

Elle devient détachée lorsqu'elle est sérialisée

Entity Manager et Persistence context

Persistence context

- représente un ensemble d'entités gérées à l'exécution
- dire qu'une entité est gérée (dans l'état *managed*) signifie qu'elle appartient à un **persistence context** donné
- les entités qui appartiennent à un **persistence context** ont un comportement commun

Entity manager

- il effectue les opérations du cycle de vie sur les entités
- il gère un **persistence context**

Entity Manager et Persistence Unit

Toutes les entités gérées par un Entity Manager sont définies grâce à une unité de persistance (*Persistence Unit*)

Une unité de persistance définit

► toutes les entités

```
<persistence version="1.0" xmlns="http://java.sun.com/xml/ns/persistence">
  <persistence-unit name="testPU" transaction-type="RESOURCE_LOCAL">
    <provider>
      oracle.toplink.essentials.ejb.cmp3.EntityManagerFactoryProvider
    </provider>
    <class>s3.Client</class>
    <properties>
      <property name="toplink.jdbc.url" value="jdbc:mysql://localhost:3306/testDB"/>
      <property name="toplink.jdbc.user" value="root"/>
      <property name="toplink.jdbc.driver" value="com.mysql.jdbc.Driver"/>
      <property name="toplink.jdbc.password" value=""/>
      <property name="toplink.target-database" value="MySQL5"/>
      <property name="toplink.ddl-generation" value="create-tables"/>
    </properties>
  </persistence-unit>
</persistence>
```

JPQL version 2.1

Support des nouveaux mappings (collections)

Opérateurs et fonctions dans les SELECT

Expressions *case*, *coalesce*, *nullif*

Possibilité d'agir sur le polymorphisme

Paramètres pour les collections dans les expressions IN

JPQL version 2.1 (cont.)

Nouveaux opérateurs

- ▶ INDEX pour les listes ordonnées

```
SELECT t FROM CreditCard c JOIN c.transactions t  
WHERE c.cardHolder.name = 'John Doe' AND INDEX(t) < 10
```

- ▶ KEY, VALUE, ENTRY pour les Maps

```
// Stock est de la forme Map<Film, Integer>  
SELECT v.emplacement.rue, KEY(i).titre, VALUE(i)  
FROM VideoStore v JOIN v.stock i  
WHERE KEY(i).metteurEnScene LIKE '%Hitchcock%' AND VALUE(i) > 0
```

- ▶ CASE, COALESCE, NULLIF pour les expressions case

```
UPDATE Employe e SET e.salaire =  
CASE e.evaluation  
  WHEN 1 THEN e.salaire * 1.05  
  WHEN 2 THEN e.salaire * 1.02  
  ELSE e.salaire * 0.95  
END
```

JPQL version 2.1 (cont.)

- ▶ TYPE pour restreindre les choix dûs au polymorphisme

```
SELECT e FROM Employe e  
WHERE TYPE(e) IN :empTypes
```

JPQL version 2.1 (cont.)

API Criteria

API pour construire dynamiquement des requêtes

Typage fort

Utilisation des *generics* de Java

Supports de la navigation basée sur les objets ou les chaînes de caractères

```
Query q = em.createQuery("SELECT a FROM Comptes");
```

```
QueryBuilder qb = em.getQueryBuilder();  
CriteriaQuery cq = qb.create();  
Root<Compte> compte = cq.from(Compte.class);  
cq.select(compte).where(...).orderBy(...);  
Query q = em.createQuery(cq);  
List<Compte> liste = q.getResultList();
```

Transactions

Types de transactions

Il existe 2 types de transactions

- ▶ transactions locales à une ressource
 - fournies par JDBC
 - associées à une seule base de données
- ▶ transactions JTA
 - plus de fonctionnalités que les transactions JDBC
 - elles peuvent travailler avec plusieurs bases de données

Dans JavaSE, transactions locales

Entity transaction

Si on n'utilise pas un serveur d'applications (par ex. Glassfish), on doit utiliser l'interface `javax.persistence.EntityTransaction`

Une instance de `EntityTransaction` s'obtient par appel de la méthode `getTransaction()` en `EntityManager`

```
public interface EntityTransaction {  
    public void begin();  
    public void commit();  
    public void rollback();  
    public void setRollbackOnly();  
    public boolean getRollbackOnly();  
    public void isActive();  
}
```

Exemple

```
EntityManager em;  
...  
try{  
    em.getTransaction().begin()  
    ...  
    em.getTransaction().commit();  
} finally{  
    em.close();  
}
```

Rollback/commit

En cas de **rollback**

- ▶ rollback dans la base de données
- ▶ le contexte de persistance est vidé, i.e. toutes les entités sont détachées
- ▶ les instances d'entités Java gardent les valeurs qu'elles avaient au moment du rollback
- ▶ le plus souvent, il faut relancer les requêtes pour éviter d'avoir des entités avec des valeurs incorrectes

En cas de **commit**

- ▶ les modifications effectuées sur les entités gérées sont enregistrées dans la base de données
- ▶ les modifications sont enregistrées dans la base, même si elles ont été effectuées avant le début de la transaction

JPA version 2.1: concurrence

Dans JPA 1.0, verrouillage optimiste

Dans JPA 2.0, possibilité d'obtenir un verrouillage pessimiste

- moins performant mais plus sûr !

Nouveaux modes de verrouillage:

- OPTIMISTIC (= READ)
- OPTIMISTIC_FORCE_INCREMENT (= WRITE)

- PESSIMISTIC_READ

- ```
public void calculCharges() {
 Compte cpt = em.find(Compte.class, cptId);
 // calcul des charges
 int charge = ... ;
 if (charge > 0) {
 em.refresh(cpt, PESSIMISTIC_WRITE);
 double solde = cpt.getSolde();
 cpt.setSolde(solde - charge);
 }
}
```

# JPA version 2.1: cache

Gestion fine du cache L2 (absent de JPA 1)

- important pour les serveurs d'applications

Fournir des données sans induire un accès à la base

- configurable: durée de vie, stratégie lecture/écriture,...

Dans JPA 2

- cache L1 activé par défaut (niveau EntityManager)
  - assure la cohérence d'une donnée au sein d'une session
- cache L2 activable à volonté (niveau EntityManagerFactory)
  - résultats de requêtes
  - requêtes
- possibilité de bypasser le cache si besoin (USE/BYPASS/REFRESH)
- possibilité de contrôler le cache (invalider tout ou partie du cache)